



APLICACIÓN WEB CON HIBERNATE

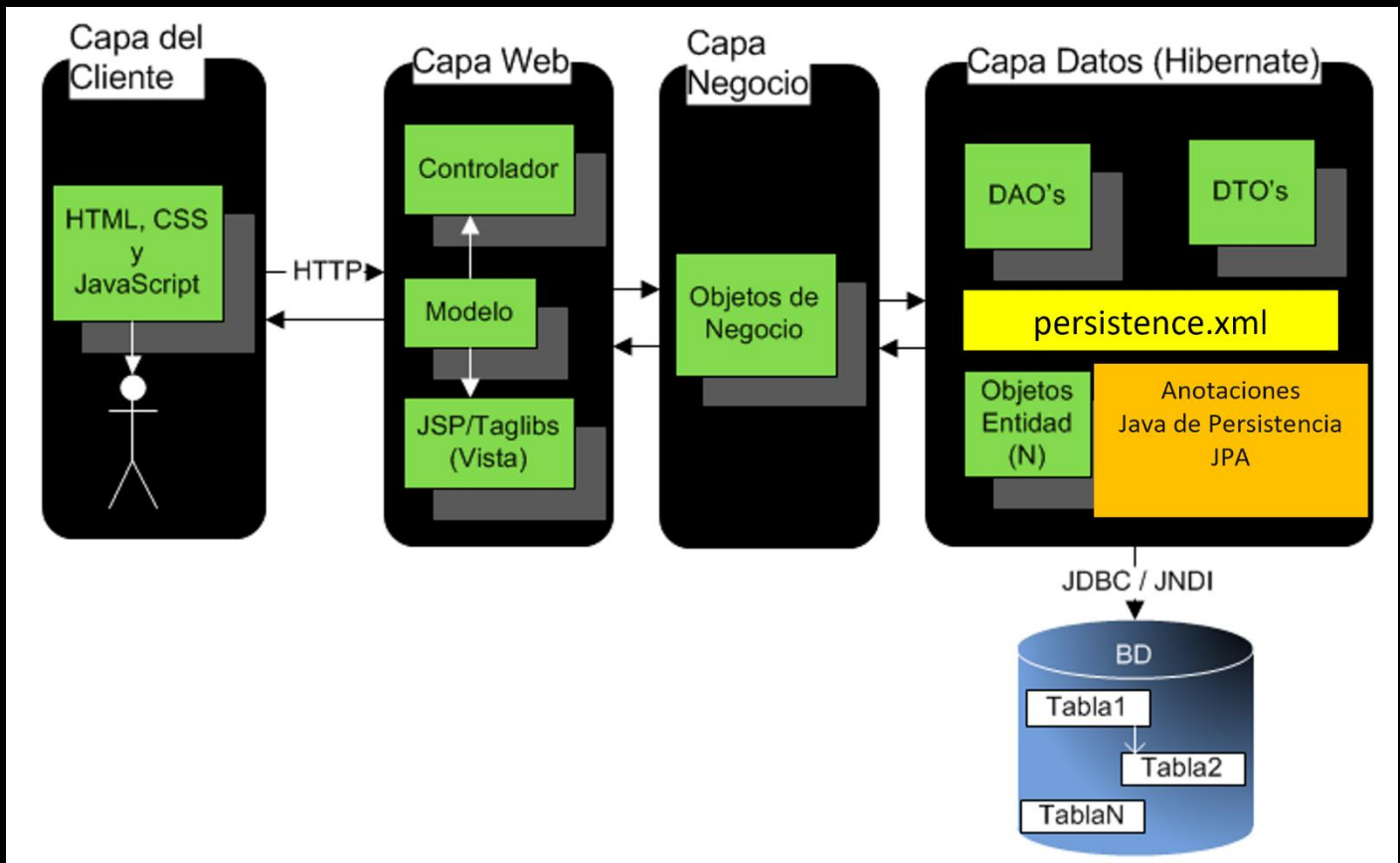
Aplicación Web: Listado de Personas con Hibernate y JPA (Parte 2)

Objetivo

Crear la capa de servicio, el servlet controlador y el JSP que permita desplegar una tabla con el listado de personas usando Hibernate y JPA.

Estructura del proyecto

Estamos construyendo una aplicación basada en arquitectura en capas:



```

Cliente (HTML + JSP)
↓
Controlador (Servlet)
↓
Servicio (Clase ServicioPersonas)
↓
DAO (Clase PersonaDAO)
↓
Entidad (Clase Persona)
↓
Base de Datos (MySQL)

```

👉 Paso 1: Crear clase ServicioPersonas

1. Clic derecho sobre Source Packages.
2. Selecciona New > Java Class.
3. Nombre de la clase: ServicioPersonas
4. Paquete: gm.servicio
5. Haz clic en Finish.

✂ Código completo de `ServicioPersonas.java`

```
package gm.servicio;

import java.util.List;
import gm.dao.PersonaDAO;
import gm.domain.Persona;

public class ServicioPersonas {

    private final PersonaDAO personaDao;

    public ServicioPersonas(){
        this.personaDao = new PersonaDAO();
    }

    public List<Persona> listarPersonas(){
        return this.personaDao.listar();
    }
}
```

Esta clase representa la **capa de servicio**, que encapsula el acceso a la capa DAO. En este ejemplo sólo agregamos el método `listarPersonas`, pero podrías agregar `insertar`, `modificar` y `eliminar`.

Paso 2: Crear el Servlet `ServletControlador`

1. Clic derecho en `Source Packages`.
2. Selecciona `New > Java Class`.
3. Nombre: `ServletControlador`
4. Paquete: `gm.web`
5. Finaliza y modifica la clase para convertirla en un Servlet.

Código completo de `ServletControlador.java`

```
package gm.web;

import java.io.IOException;
import java.util.List;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import gm.domain.Persona;
import gm.servicio.ServicioPersonas;
```

```
@WebServlet("/ServletControlador")
public class ServletControlador extends HttpServlet{
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response){
        ServicioPersonas servicioPersonas = new ServicioPersonas();

        List<Persona> personas = servicioPersonas.listarPersonas();

        request.setAttribute("personas", personas);

        try {
            request.getRequestDispatcher("/WEB-INF/listado.jsp").forward(request, response);
        } catch (ServletException | IOException ex) {
            ex.printStackTrace(System.out);
        }
    }
}
```

Este servlet actúa como **controlador**: recupera datos, los guarda en `request` y redirige al JSP.

Paso 3: Modificar `index.html`

Abre el archivo `index.html` y reemplaza su contenido por:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Ejemplo Personas Web Hibernate JPA</title>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    </head>
    <body>
        <h1>Ejemplo Personas Web Hibernate JPA!</h1>
        <br/>
        <a href="/PersonasWebHibernateJPA/ServletControlador">Listar Personas con Hibernate y JPA</a>
    </body>
</html>
```

Asegúrate de escribir correctamente el nombre de la aplicación y la ruta del servlet (`ServletControlador`), respetando mayúsculas y minúsculas.

Paso 5: Crear el archivo JSP `listado.jsp`

1. Navega a la carpeta `Web Pages/WEB-INF`.
2. Clic derecho > New > JSP.
3. Nombre: `listado`
4. Haz clic en **Finish**.

Código completo de `listado.jsp`

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<!DOCTYPE html>
<html>
  <head>
    <title>Listado de Personas</title>
  </head>
  <body>
    <table border="1">
      <caption>Listado de Personas</caption>
      <tr>
        <th>Id Persona</th>
        <th>Nombre</th>
        <th>Apellido</th>
        <th>Email</th>
        <th>Teléfono</th>
      </tr>
      <c:forEach var="persona" items="${personas}" >
        <tr>
          <td>${persona.idPersona}</td>
          <td>${persona.nombre}</td>
          <td>${persona.apellido}</td>
          <td>${persona.email}</td>
          <td>${persona.telefono}</td>
        </tr>
      </c:forEach>
    </table>

  </body>
</html>
```

Este JSP **itera** sobre la lista de personas usando JSTL y muestra los datos en una tabla HTML.

Paso 6: Solución de errores comunes

Si al ejecutar la aplicación ocurre un error como:

Public Key Retrieval is not allowed

👉 Agrega este parámetro `allowPublicKeyRetrieval=true` a tu URL de conexión en `persistence.xml`:

```
<property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=false&useTimezone=true&serverTimezone=UTC&allowPublicKeyRetrieval=true"/>
```

Esto es necesario desde las versiones recientes del conector MySQL.

📄 Código completo `persistence.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
    https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

  <persistence-unit name="HibernatePU" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>gm.domain.Persona</class>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=false&useTimezone=true&serverTimezone=UTC&allowPublicKeyRetrieval=true"/>
      <property name="jakarta.persistence.jdbc.user" value="root"/>
      <property name="jakarta.persistence.jdbc.password" value="admin"/>
      <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
    </properties>
  </persistence-unit>
</persistence>
```

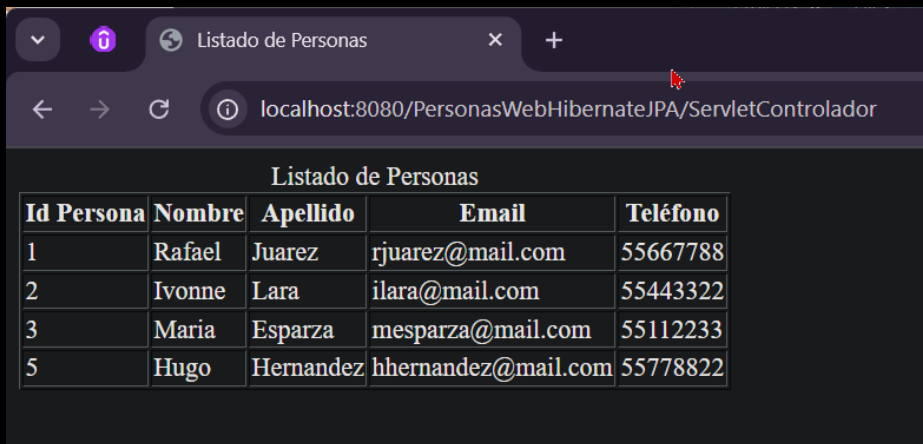
Paso 7: Verificar y ejecutar

1. Haz clic derecho en el proyecto > **Run**.
2. Asegúrate de que el servidor GlassFish esté activo.
3. En el navegador accede a:

<http://localhost:8080/PersonasWebHibernateJPA/>



4. Da clic en el enlace **Listar Personas**.
5. Verifica que se despliegue la tabla con tus registros.



Conclusión

🎉 ¡Tu aplicación web ya muestra el listado de personas correctamente!

- ✓ Aplicaste arquitectura en capas
- ✓ Integraste Hibernate y JPA
- ✓ Mostraste los datos desde base de datos en un JSP

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)